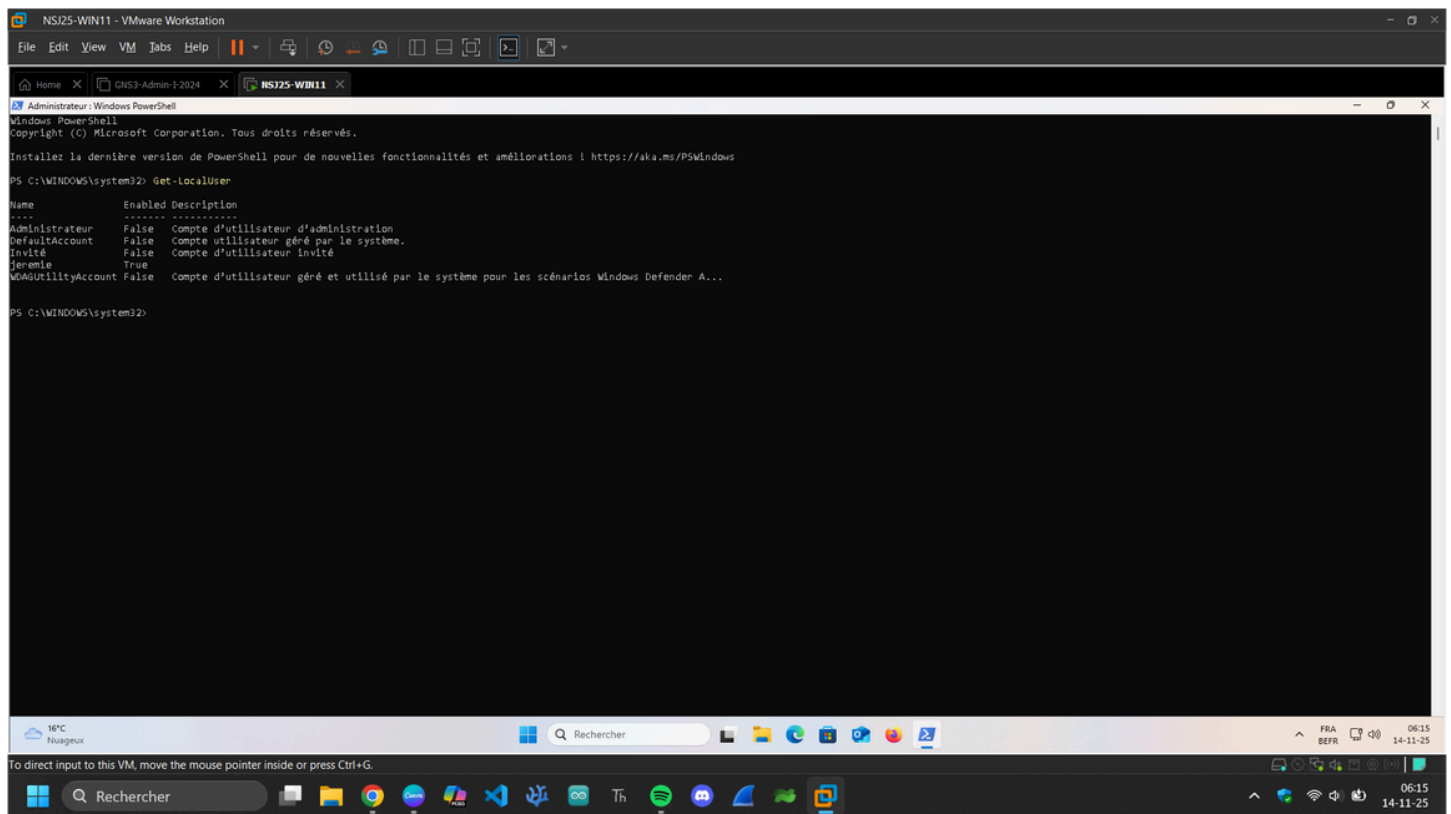


2 Prise en main

2.1 Lister les utilisateurs existants

Dans un PowerShell en mode Administrateur, listez l'ensemble des utilisateurs locaux de la machine.

Get-LocalUser



The screenshot shows a Windows PowerShell window titled "Administrateur : Windows PowerShell" running the command `Get-LocalUser`. The output lists local users with their names, enabled status, and descriptions.

Name	Enabled	Description
Administrateur	False	Compte d'utilisateur d'administration
DefaultAccount	False	Compte utilisateur géré par le système.
Invité	False	Compte d'utilisateur invité
Jeremie	True	
WdAGUtilityAccount	False	Compte d'utilisateur généré et utilisé par le système pour les scénarios Windows Defender A...

Qu'est-ce qu'un compte désactivé ?

Un **compte désactivé** = un compte **qui existe**, mais **qui ne peut pas se connecter** à l'ordinateur.

Impossible d'ouvrir une session avec lui mais il reste visible dans la liste des comptes

On peut le réactiver à tout moment avec :

Enable-LocalUser -Name "NomDuCompte"

Le compte Administrator est désactivé par défaut sur Windows 11 pour une bonne raison, laquelle ?

Parce que ce compte est **extrêmement puissant** :

Il a **tous les droits sans aucune restriction**

Il n'est **pas soumis au contrôle UAC** (pas de confirmation de sécurité)

S'il restait activé, il deviendrait une cible facile pour les pirates → ils connaissent déjà son nom exact : **"Administrator"**

Raison principale : sécurité

Un pirate qui obtient le mot de passe ou corrompt le système pourrait tout contrôler sans limite.

Donc Microsoft désactive ce compte pour éviter :

- les intrusions,
- les malwares qui pourraient l'utiliser,
- les mauvaises manipulations.

Windows préfère que tu utilises un **compte administrateur standard**, qui lui est protégé par l'UAC.

3 Création et configuration d'un utilisateur

Nous allons créer un compte pour un nouvel employé fictif, "Bob Marley". Pour ce faire, nous avons besoin de plusieurs informations : son identité complète,

un identifiant utilisé par la machine, un mot de passe, et une description si on le souhaite.

Nom complet : Bob Marley

Identifiant : bmarley

Password : bMleM8lleur !

Description : Un magnifique chanteur !

3.1 Comprendre les aspects sécurité des mots de passe

Avant de commencer, il est important d'aborder certains concepts de sécurité.

Quelle est la stratégie de Microsoft en terme de mot de passe ?

La stratégie actuelle de Microsoft repose sur un principe important :

Objectif : améliorer la sécurité en réduisant la dépendance au mot de passe.

Microsoft considère aujourd'hui que les mots de passe sont :

- faciles à oublier,
- souvent réutilisés,
- vulnérables au phishing, au bruteforce, et aux fuites.

La stratégie officielle : « Passwordless » (sans mot de passe)

Microsoft encourage l'abandon progressif des mots de passe au profit de méthodes plus sûres :

Windows Hello (empreinte, PIN, reconnaissance faciale)

Microsoft Authenticator

Clés de sécurité FIDO2

Authentification multi-facteurs (MFA)

Ces méthodes sont plus sécurisées parce que :

- elles ne peuvent pas être volées via phishing,
- elles sont propres à l'appareil ou au propriétaire,
- elles ne circulent pas sur le réseau.

Microsoft ne recommande plus :

les mots de passe complexes obligatoires (ex : fH!8@k9%)

les expirations régulières forcées

Ces mesures poussent souvent les utilisateurs à choisir des mots de passe **moins sécurisés** ou à les écrire quelque part.

Que fait cette ligne de commande ?

\$Password = Read-Host -AsSecureString -Prompt "Entrez un mot de passe"

Cette commande **demande à l'utilisateur d'entrer un mot de passe dans PowerShell**, sans l'afficher en clair à l'écran.

Le mot de passe saisi est ensuite **stocké dans la variable \$Password, sous forme sécurisée (SecureString)**.

Que représente chaque paramètre ?

\$Password =

Crée une variable appelée **\$Password**.

Elle va recevoir le résultat de la commande Read-Host.

Donc : **le mot de passe saisi par l'utilisateur.**

Read-Host

C'est une commande PowerShell qui sert à **demander une entrée utilisateur**, comme un input.

Elle affiche un message, attend que l'utilisateur tape quelque chose, puis retourne la valeur.

-Prompt "Entrez un mot de passe"

C'est le **texte affiché à l'écran** pour guider l'utilisateur.

-AsSecureString

Le paramètre le plus important.

Il demande à PowerShell de stocker le mot de passe sous forme de **SecureString** :

- le texte saisi n'est **pas affiché** (aucun caractère visible),
- la valeur est **chiffrée en mémoire**,
- elle ne peut pas être lue facilement ou affichée en clair.

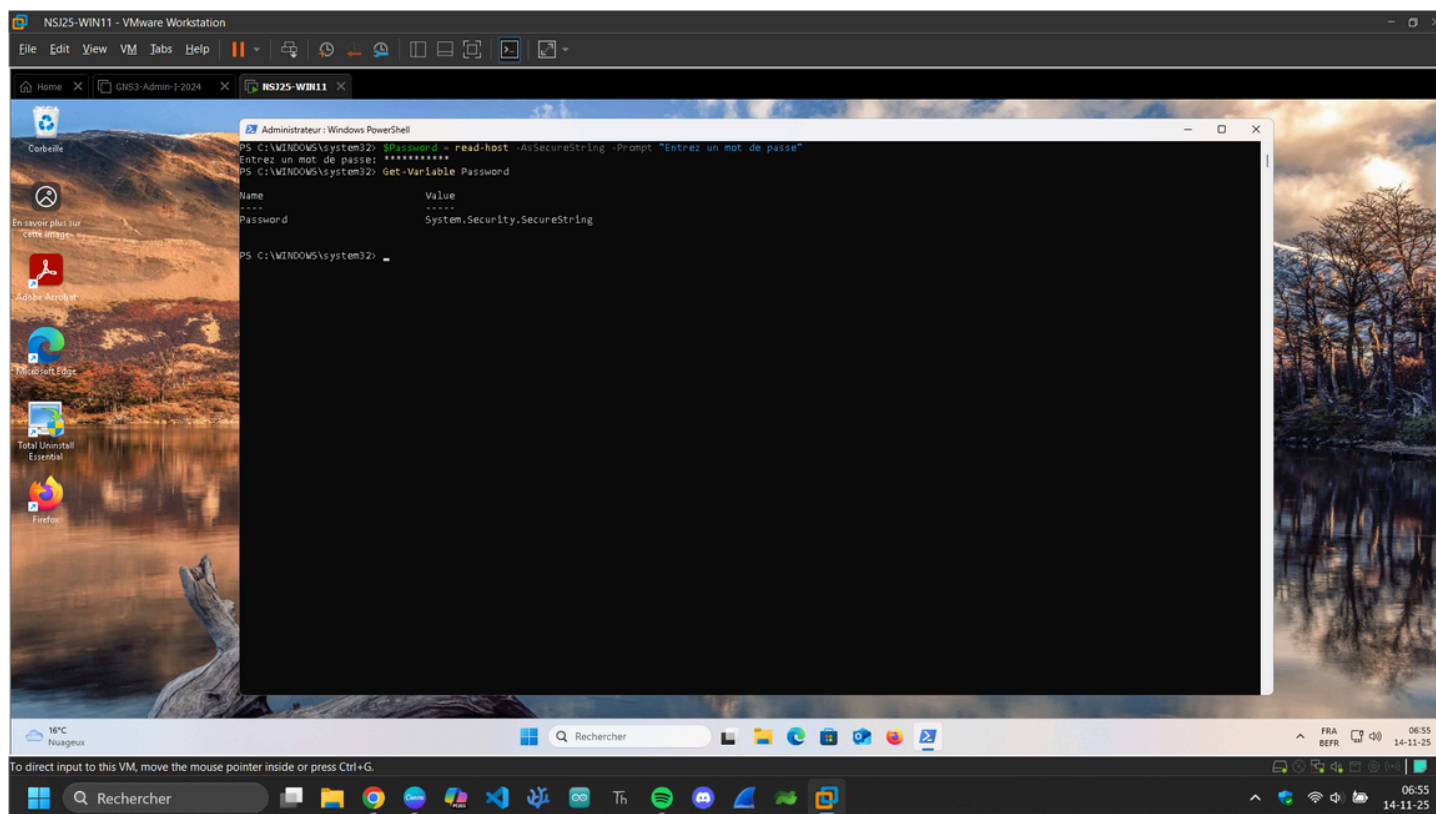
Sans ce paramètre, le mot de passe serait visible et en clair, ce qui serait **dangereux**.

Que fait cette ligne de commande ?

Get-Variable Password

Cette commande **affiche le contenu de la variable nommée Password** ainsi que ses propriétés (nom, valeur, type, etc.).

Elle sert à **voir ce qu'il y a dans la variable \$Password**.



Que représente chaque paramètre ?

Get-Variable

C'est une commande PowerShell qui permet de **recupérer une ou plusieurs variables**.

Elle affiche :

- le nom de la variable,
- sa valeur,
- son type (String, Int32, SecureString, etc.),
- son module d'origine.

Password

C'est le **nom de la variable** que tu veux afficher.

Cela correspond à \$Password.

Important : dans Get-Variable, **on ne met pas le signe \$**, on donne seulement le nom.

Qu'est-ce que la classe **System.Security.SecureString** ?

System.Security.SecureString est une classe spéciale de **.NET** (utilisée par PowerShell) qui sert à stocker du texte sensible de manière sécurisée, comme :

- un mot de passe,
 - une clé API,
 - un code secret,
 - une information confidentielle.
-

3.2 Définir les aspects sécurité indispensables

New-LocalUser -Name "bmarley" -Password \$Password -FullName "Bob Marley" -
Description "Un magnifique chanteur !"

Après la création de l'utilisateur, il est important de configurer les politiques de sécurité pour ce compte. Dans un cas contraire, quels sont les risques liés à ce compte tout juste créé avec ces informations ?

Mot de passe faible → piratage facile

Si tu n'imposes pas une politique concernant :

- la longueur minimale,
- la complexité,
- l'historique,
- les tentatives de connexion (lockout),

alors l'utilisateur peut mettre un mot de passe du style :

1234, password, bob, marley etc...

Risque :

- piratage par **brute force**
- piratage par **devinettes simples**
- attaque par **dictionnaire**

Le mot de passe peut ne jamais expirer

Sans politique d'expiration, le mot de passe reste valable **à vie**, même s'il fuit.

Risque :

- un mot de passe compromis reste valide longtemps
- un ancien employé peut encore se connecter

L'utilisateur peut se connecter sans limitation (pas de verrouillage)

Si aucune politique de verrouillage n'est configurée :

- un attaquant peut essayer **des milliers de mots de passe**
- le compte ne se bloque jamais

Risque :

- attaques bruteforce illimitées

Le compte peut être utilisé pour des activités malveillantes

Si le mot de passe a été volé ou mal choisi, l'attaquant peut :

- installer des programmes,
- voler des données,
- accéder au réseau local,
- contourner la sécurité du système s'il obtient plus de droits ensuite.

Le compte n'est pas protégé par une stratégie d'audit

Sans journalisation :

- Impossible de savoir **qui s'est connecté**
- Impossible de détecter une **intrusion**
- Impossible de savoir quand le mot de passe a été changé

C'est un risque énorme dans un environnement pro.

Informations personnelles dans la description

Le champ :

"Un magnifique chanteur !"

Donne **une information non professionnelle** et identifiable.

Risque :

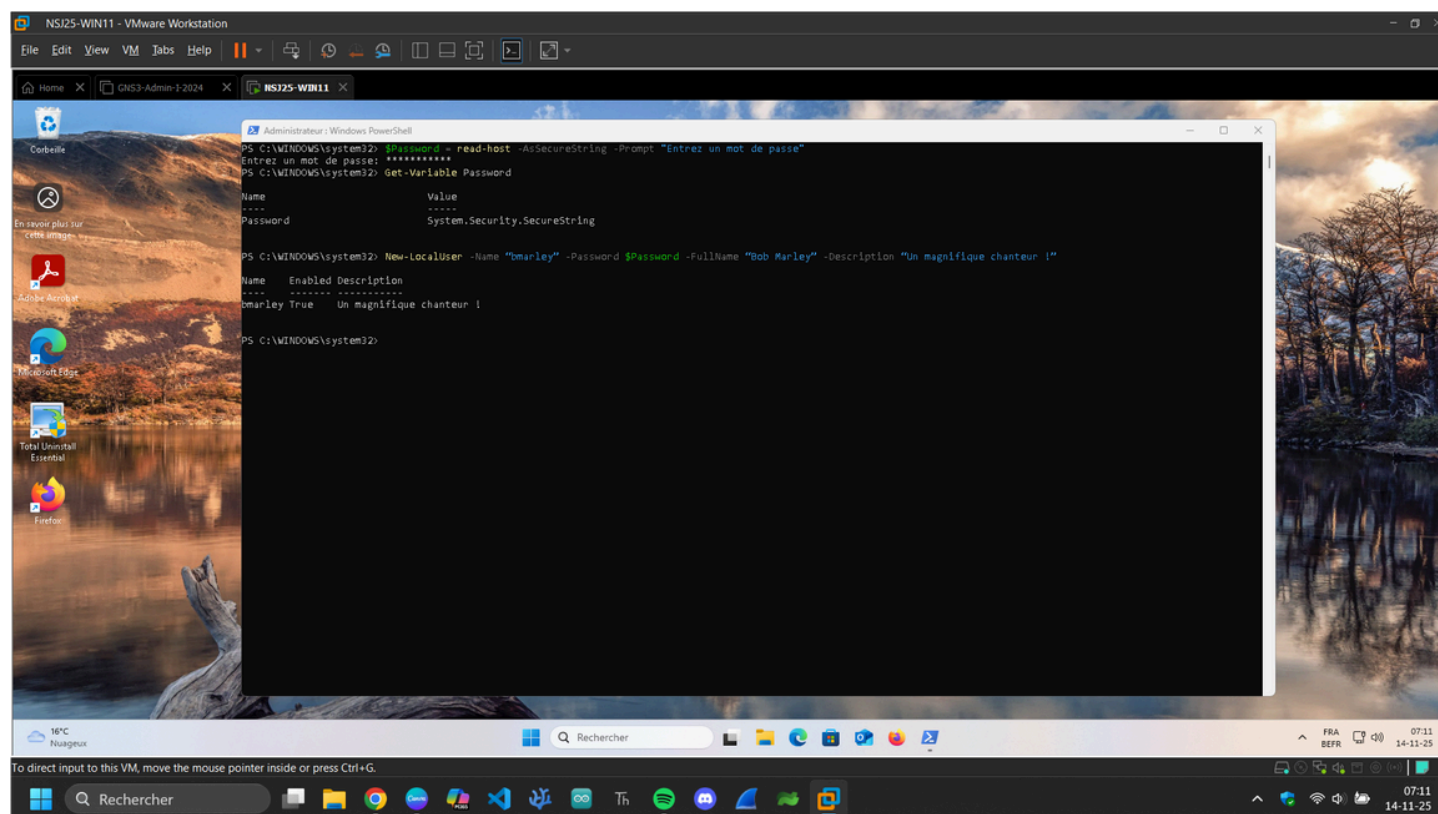
- fuite de données personnelles
- manque de conformité (RGPD) dans un cadre entreprise
- infos inutiles pour un compte utilisateur

Le mot de passe peut être stocké en clair dans les scripts

Si quelqu'un a mis \$Password en clair au lieu de SecureString (courant chez les débutants), l'administrateur pourrait :

- laisser un mot de passe dans un script,
- laisser un mot de passe dans l'historique PowerShell,
- laisser un mot de passe dans un fichier .ps1.

Très gros risque de fuite.



3.3 Analyse de ce nouveau compte

Rendez-vous dans l'interface de gestion des utilisateurs et des groupes et analysons les propriétés du compte bmarley.

Méthode 1 : Avec lusrmgr.msc (la plus rapide)

1. Appuie sur **Windows + R**
2. Tape : **lusrmgr.msc**
3. Appuie sur **Entrée**

Tu arrives directement dans **Utilisateurs et groupes locaux**.

Méthode 2 : Via le menu Démarrer

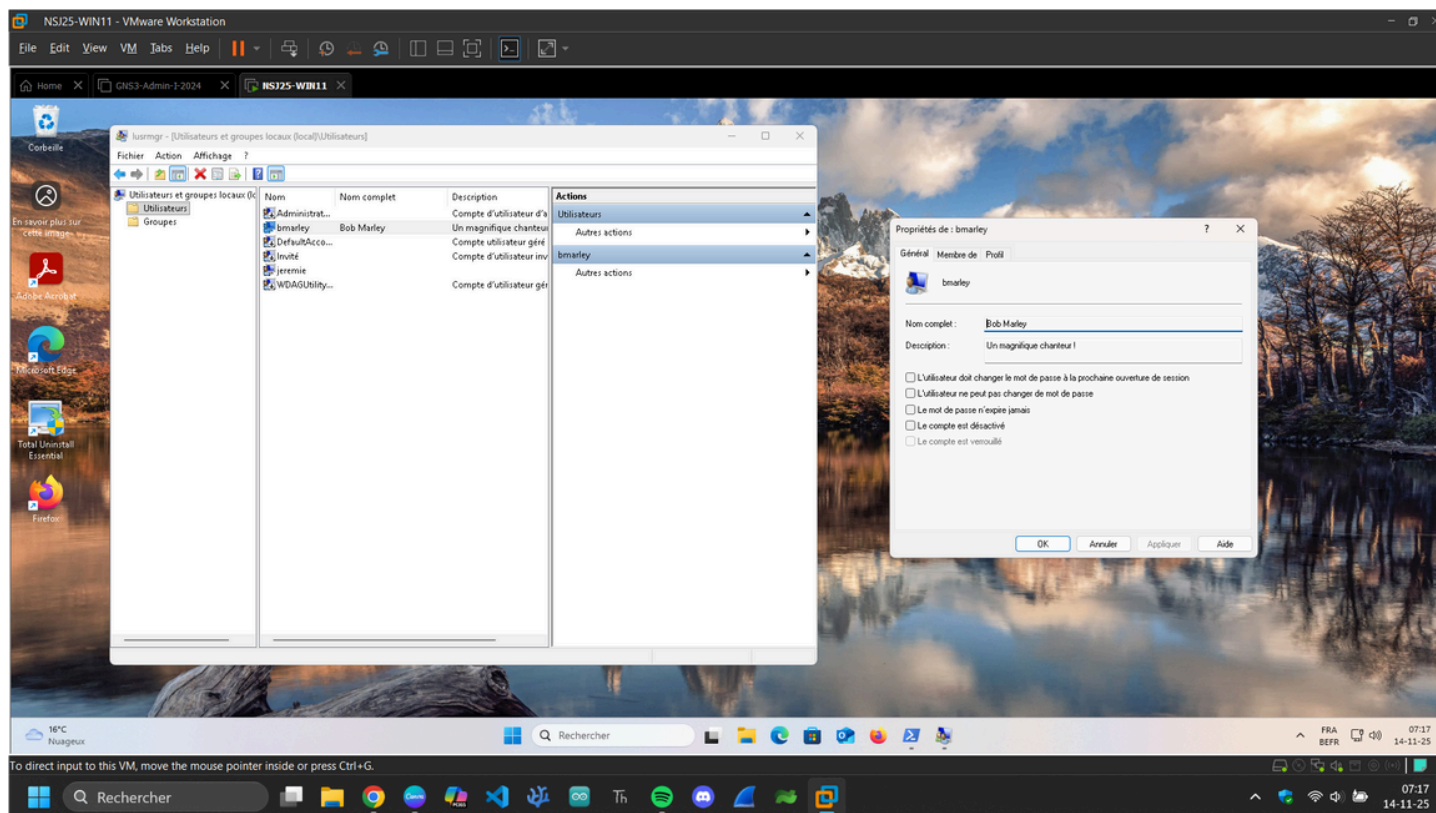
1. Clique sur **Démarrer**
2. Tape "**lusrmgr**" dans la recherche
3. Clique sur **Utilisateurs et groupes locaux**

Méthode 3 : Via la gestion de l'ordinateur

1. Clique droit sur **Ce PC**
2. Choisis **Gérer**

3. Dans la colonne de gauche, vas dans :

- **Outils système**
- **Utilisateurs et groupes locaux**



Au sinon, vous pouvez aussi accéder à ces informations via le terminal en utilisant la commande

```
Get-LocalUser -Name "bmarley" | Format-List *
```

Que remarquez-vous dans les propriétés de l'utilisateur ?

Quand tu ouvres **bmarley** → **Propriétés**, tu constates généralement :

Le mot de passe n'expire jamais (Case Password never expires souvent cochée après une création PowerShell)

L'utilisateur n'est pas obligé de changer son mot de passe au premier logon (Case User must change password at next logon non cochée)

Aucune restriction d'accès :

- Pas de limite d'horaires (*Logon hours*)

- Pas de restriction de machine (*Log on to*)
- Le compte est actif 24h/24

Description inutile / non professionnelle → « Un magnifique chanteur ! »

Aucune configuration de profil ou chemin d'ouverture de session

En résumé : Le compte est fonctionnel... mais **pas du tout sécurisé**.

Est-ce que les politiques de sécurité les plus fondamentales sont activées ?

Non. Elles ne sont PAS activées.

Les politiques de base comme :

- **Mot de passe doit être changé à la première connexion**
- **Expiration du mot de passe**
- **Heures de connexion limitées**
- **Restrictions de machines autorisées**
- **Verrouillage du compte après tentatives ratées**

Rien n'est configuré par défaut sur un compte créé via PowerShell.

Lesquelles devraient l'être ?

Les politiques minimum recommandées :

Obliger l'utilisateur à changer son mot de passe à la première connexion Évite qu'il garde un mot de passe que l'admin connaît.

- **Expiration du mot de passe** → Renouvellement tous les X jours (selon la politique locale)
- **Verrouillage en cas de tentatives incorrectes** → Empêche les attaques bruteforce.
- **Interdire les mots de passe trop faibles** → S'applique via les stratégies locales de sécurité.
- **Définir des heures de connexion** → Par exemple : pas de connexion durant la nuit.
- **Limiter les machines sur lesquelles l'utilisateur peut se connecter** → Surtout dans un environnement entreprise.

De quel groupe d'utilisateurs fait-il partie ?

Par défaut, un compte créé avec :

New-LocalUser

est ajouté automatiquement au groupe :

Users (Utilisateurs)

Il n'est **PAS** membre de Administrators et c'est **une bonne chose**, car cela limite fortement les dégâts en cas de piratage.

Règlons tout cela via le terminal

```
Set-LocalUser -Name "bmarley" -PasswordNeverExpires $False -  
UserMayChangePassword $True
```

Cette commande **modifie les propriétés du compte local "bmarley"** afin d'appliquer des politiques de sécurité plus strictes.

Elle change deux choses importantes :

A quoi servent chacun des paramètres ?

Set-LocalUser

Commande PowerShell utilisée pour **modifier** un utilisateur local existant.

```
-Name "bmarley"
```

Indique **quel utilisateur** tu veux modifier.

Ici : le compte **bmarley**.

```
-PasswordNeverExpires $False
```

Définit que le mot de passe **ne doit pas être permanent**.

\$False = **désactivé**, donc :

Le mot de passe devra expirer selon la stratégie locale de sécurité.

Objectif sécurité : obliger un changement régulier du mot de passe.

-UserMayChangePassword \$True

Autorise l'utilisateur à **changer son mot de passe lui-même**.

\$True = activé

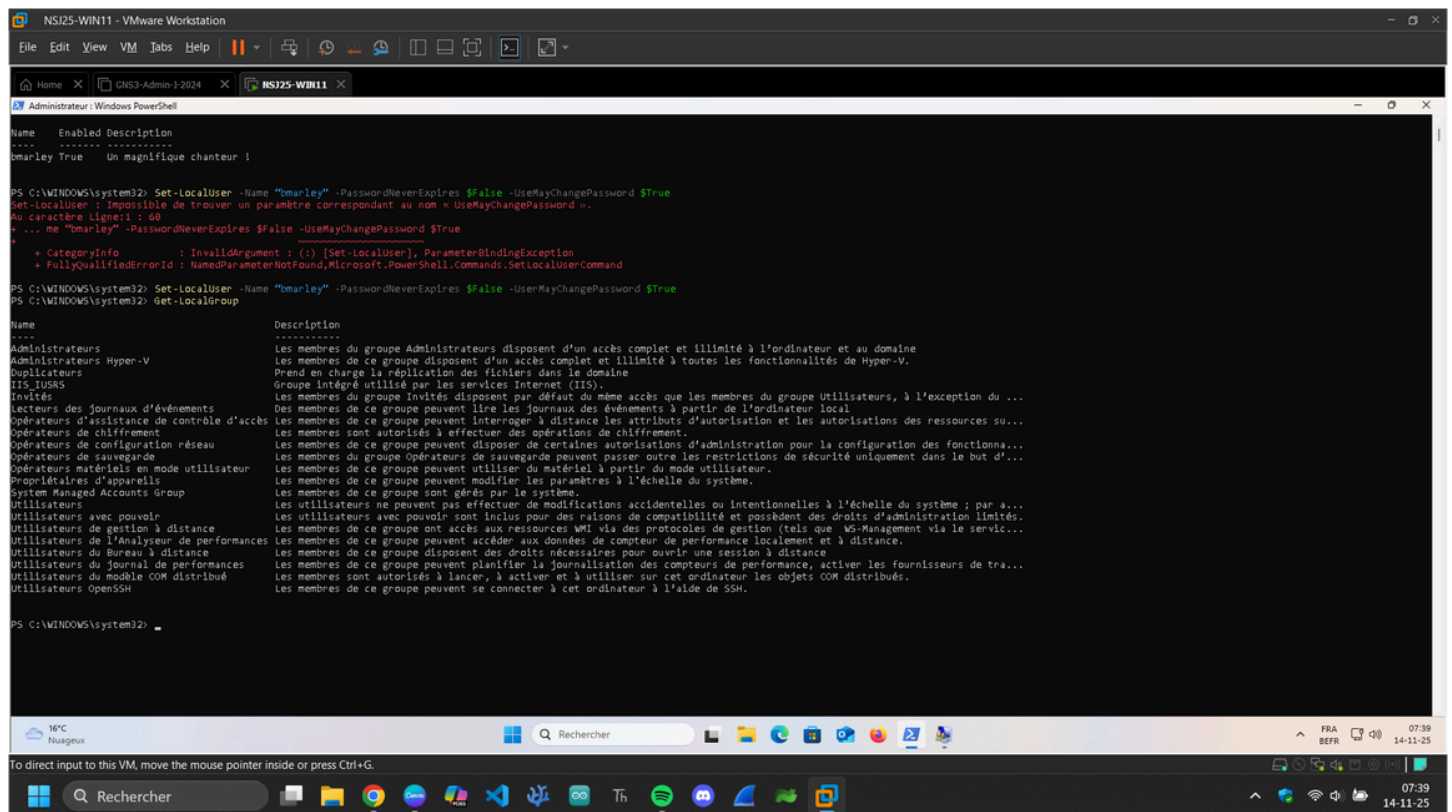
Sans ça, l'utilisateur resterait coincé avec un mot de passe choisi par l'admin → très risqué.

3.4 Définir des permissions

Les permissions sont rarement données directement à un utilisateur mais à un groupe. C'est ensuite à ce groupe que sera affilié le nouvel utilisateur.

3.4.1 Lister les groupes locaux

Get-LocalGroup



```
PS C:\WINDOWS\system32> Set-LocalUser -Name "bmarley" -PasswordNeverExpires $False -UserMayChangePassword $True
Set-LocalUser : Impossible de trouver un paramètre correspondant au nom « UserMayChangePassword ».
Au caractère Ligne:1 : 60
+ ... ne "bmarley" -PasswordNeverExpires $False -UserMayChangePassword $True
+ ~~~~~
+ CategoryInfo          : InvalidArgument ( (:) [Set-LocalUser], ParameterBindingException
+ FullyQualifiedErrorId : NamedParameterNotFound,Microsoft.PowerShell.Commands.SetLocalUserCommand

PS C:\WINDOWS\system32> Set-LocalUser -Name "bmarley" -PasswordNeverExpires $False -UserMayChangePassword $True
PS C:\WINDOWS\system32> Get-LocalGroup

Name                           Description
-----
Administrateurs                 Les membres du groupe Administrateurs disposent d'un accès complet et illimité à l'ordinateur et au domaine
Administrateurs Hyper-V        Les membres de ce groupe disposent d'un accès complet et illimité à toutes les fonctionnalités de Hyper-V.
Duplicateurs                   Prend en charge la répliquation des fichiers dans le domaine
IIS_IUSRS                      Groupe intégré utilisé par les services Internet (IIS).
Invités                        Les membres du groupe Invités disposent par défaut du même accès que les membres du groupe Utilisateurs, à l'exception du ...
Lecteurs des journaux d'événements Les membres de ce groupe peuvent lire les journaux des événements à partir de l'ordinateur local
Opérateurs d'assistance de contrôle d'accès Les membres de ce groupe peuvent interroger à distance les attributs d'autorisation et les autorisations des ressources su...
Opérateurs de chiffrement      Les membres sont autorisés à effectuer des opérations de chiffrement.
Opérateurs de configuration réseau Les membres de ce groupe peuvent disposer de certaines autorisations d'administration pour la configuration des fonctionna...
Opérateurs de sauvegarde        Les membres du groupe Opérateurs de sauvegarde peuvent passer outre les restrictions de sécurité uniquement dans le but d'...
Opérateurs matériels en mode utilisateur Les membres de ce groupe peuvent utiliser du matériel à partir du mode utilisateur.
Propriétaires d'appareils       Les membres de ce groupe peuvent modifier les paramètres à l'échelle du système.
System Managed Accounts Group  Les membres de ce groupe sont gérés par le système.
Utilisateurs                   Les utilisateurs ne peuvent pas effectuer de modifications accidentelles ou intentionnelles à l'échelle du système ; par d...
Utilisateurs avec pouvoir       Les utilisateurs avec pouvoir sont inclus pour des raisons de compatibilité et possèdent des droits d'administration limités.
Utilisateurs de gestion à distance Les membres de ce groupe ont accès aux ressources WMI via des protocoles de gestion (tels que WS-Management via le servic...
Utilisateurs de l'Analyseur de performances Les membres de ce groupe peuvent accéder aux données de compteur de performance localement et à distance.
Utilisateurs du Bureau à distance Les membres de ce groupe disposent des droits nécessaires pour ouvrir une session à distance.
Utilisateurs du Journal de performances Les membres de ce groupe peuvent planifier la journalisation des compteurs de performance, activer les fournisseurs de tra...
Utilisateurs du modèle COM distribué Les membres sont autorisés à lancer, à activer et à utiliser sur cet ordinateur les objets COM distribués.
Utilisateurs OpenSSH           Les membres de ce groupe peuvent se connecter à cet ordinateur à l'aide de SSH.
```

On a toujours l'impression qu'il y a peu de groupes, comme simplement Administrators et Users, mais en réalité il en existe déjà un certain nombres. Essayez de comprendre l'utilité de : Administrators, Guests, OpenSSH Users, Remote Desktop Users, et Users.

Administrators

Membres : utilisateurs ayant des privilèges complets sur la machine.

Pouvoirs :

- Installer et désinstaller des logiciels.
- Modifier toutes les configurations système.
- Créer, modifier et supprimer tous les comptes utilisateurs.
- Changer les stratégies de sécurité.

Risque : si un compte Administrators est compromis → le pirate peut tout contrôler.

Utilisation : uniquement pour l'administrateur ou les utilisateurs de confiance.

Users

Membres : utilisateurs standards.

Pouvoirs :

- Utiliser des logiciels et des applications.
- Modifier leurs propres fichiers et paramètres.
- Pas le droit de changer les paramètres système critiques.

Utilisation : tous les comptes normaux doivent être dans ce groupe.

Sécurité : limite les dégâts si le compte est compromis.

Guests

Membres : comptes temporaires ou invités.

Pouvoirs :

- Très limités : accès minimal aux fichiers et applications.
- Pas le droit de modifier les paramètres ou d'installer des logiciels.

Utilisation : pour des visiteurs qui ont besoin d'un accès temporaire.

Sécurité : sécurité renforcée car presque aucun droit.

Remote Desktop Users

Membres : utilisateurs autorisés à se connecter via **Remote Desktop** (RDP).

Pouvoirs :

- Connexion à distance uniquement.
- Les droits dépendent du groupe "Users" ou "Administrators" auquel ils appartiennent.

Utilisation : accès sécurisé pour télétravail ou support à distance.

OpenSSH Users

Membres : utilisateurs autorisés à se connecter via **SSH** si le service OpenSSH est installé.

Pouvoirs :

- Connexion sécurisée en ligne de commande depuis un autre ordinateur.
- Droits dépendant du groupe standard (Users ou Administrators).

Utilisation : administration distante via SSH.

3.4.2 Créer un groupe local pour nos nouveaux utilisateurs

New-LocalGroup -Name "Chanteurs" -Description "Comptes avec accès limites pour les chanteurs"

Cela ajoute **bmarley** au groupe **Chanteurs**, lui donnant les droits du groupe.

Add-LocalGroupMember -Group "Chanteurs" -Member "bmarley"

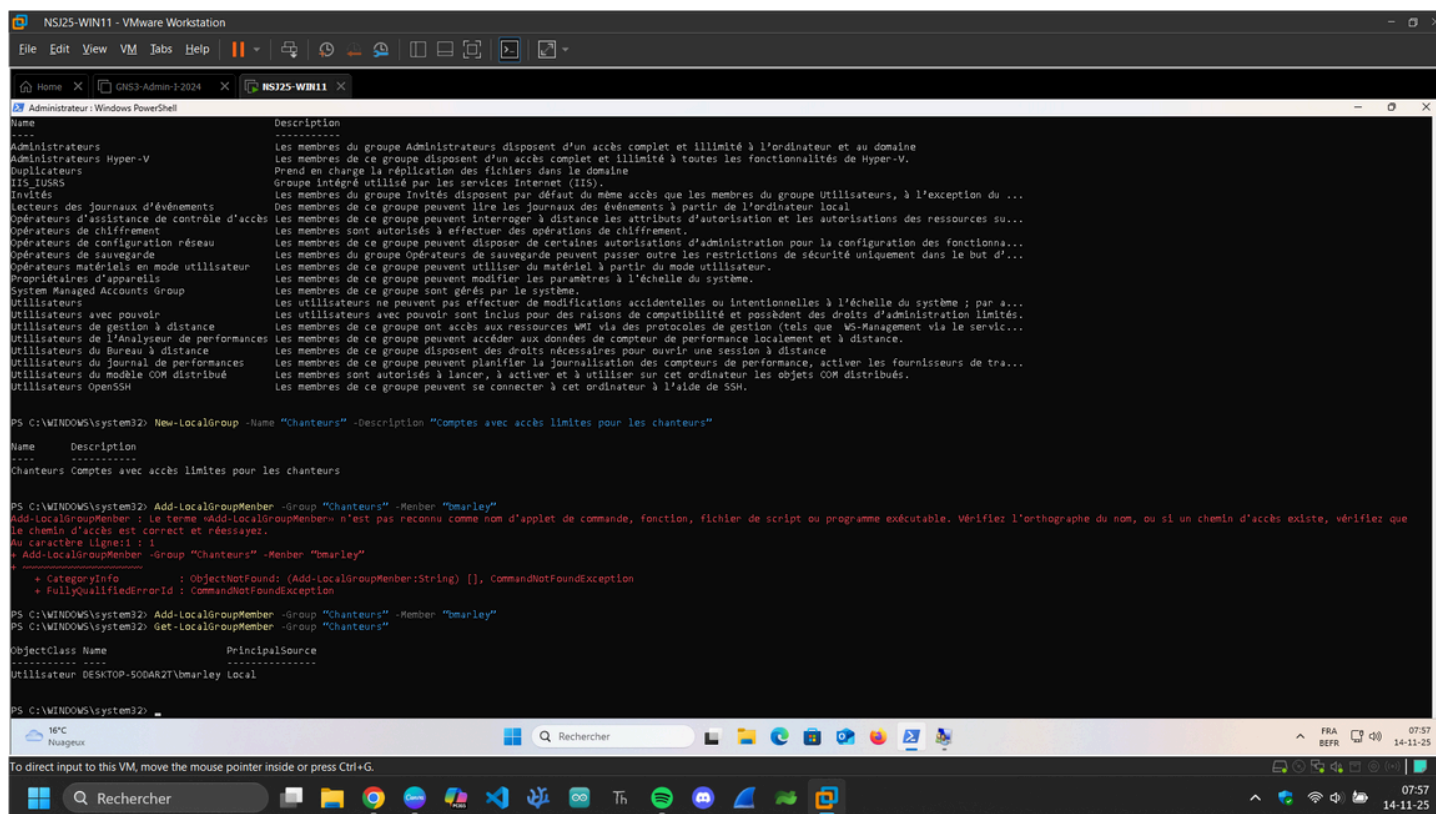
Cette commande **ajoute un utilisateur existant (bmarley) dans un groupe local (Chanteurs)**.

L'utilisateur **hérite alors des droits et restrictions du groupe**.

Get-LocalGroupMember -Group "Chanteurs"

Cette commande liste tous les membres du groupe local Chanteurs.

Si bmarley a bien été ajouté, son nom apparaîtra dans la liste.



Pourquoi est-ce que le nom de l'utilisateur dans le groupe est précédé de PC01 ?

Utilisateur **DESKTOP-50DAR2T\bmarley** Local

Dans PowerShell et dans Windows en général, quand tu regardes les **membres d'un groupe local**, tu peux avoir deux types d'utilisateurs :

Utilisateur local

Format affiché : **NomOrdinateur\NomUtilisateur**

Exemple : **DESKTOP-50DAR2T\bmarley**

- **DESKTOP-50DAR2T** → le **nom de l'ordinateur** (le domaine local de l'ordinateur)
- **bmarley** → le nom de l'utilisateur local

Cela signifie que bmarley est **un compte local sur cet ordinateur**, pas un compte réseau ou domaine.

Utilisateur de domaine

Format affiché : **NomDomaine\NomUtilisateur**

Exemple : **WOODYTOYS\bmarley** Ici, l'utilisateur fait partie d'un **domaine réseau**.

PowerShell ajoute la mention **Local** pour indiquer que c'est un **compte local de l'ordinateur**, et non un compte de domaine.

Donc, **"PC01" ou "DESKTOP-5ODAR2T" devant le nom** n'est pas un problème, c'est juste la convention pour identifier que c'est un compte local.

Est-ce qu'il serait intéressant d'ajouter "bmarley" au groupe "Administrators" ? Si non, pour quelle raison ?

Réponse : **Non**, ce n'est pas recommandé.

Les droits d'Administrators sont très étendus

Contrôle total sur la machine : installation de logiciels, modification du système, création et suppression de comptes.

Pas de limites pour l'utilisateur.

Si un compte Administrators est compromis, un pirate peut **tout contrôler**.

Risque de sécurité

bmarley est censé être un **compte standard pour un chanteur**.

Ajouter ce compte aux Administrators serait **exposer le système à des risques inutiles**.

Exemples :

Malware qui utilise le compte pour infecter le système.

Mauvaises manipulations accidentelles pouvant casser Windows.

Principe de sécurité

Il est recommandé de suivre le principe **"least privilege"** :

Chaque utilisateur ne doit avoir que les droits **strictement nécessaires** pour son rôle.

Ici, un compte normal **n'a pas besoin de privilèges d'administrateur** pour travailler.

4 Nettoyer des comptes utilisateurs

Lorsque des utilisateurs ne doivent plus avoir accès à la machine, il est important de supprimer ou désactiver le compte. Dans un idéal, et par mesure de sécurité, il est préférable de désactiver le compte et donc de bloquer l'accès à celui-ci. C'est une mesure non-destructive et temporaire, ce qui est préférable dans un premier temps.

```
Disable-LocalUser -Name "bmarley"
```

Cette commande **désactive le compte local bmarley**.

```
Get-LocalUser -Name "bmarley" | Select-Object Name, Enabled
```

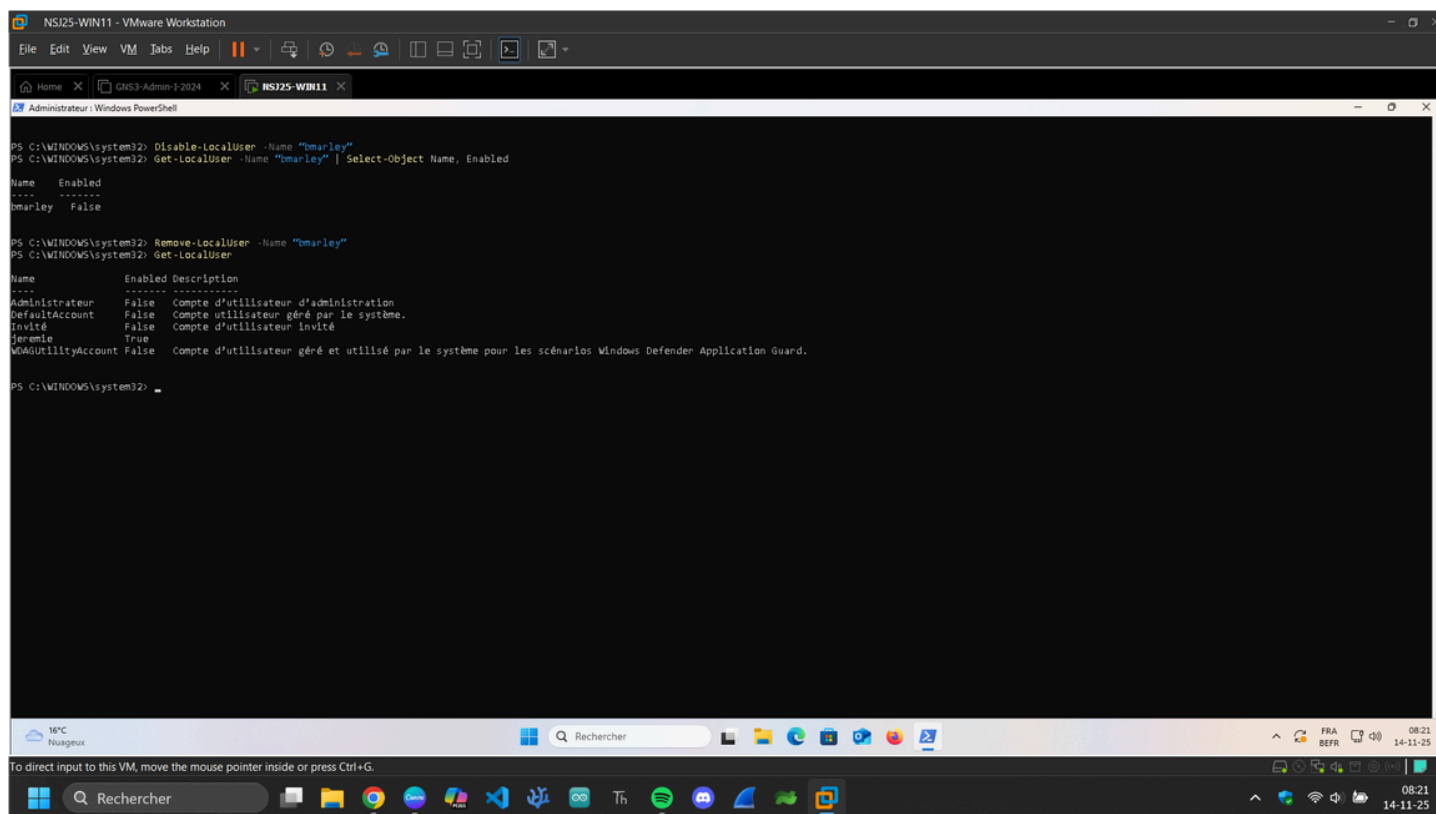
Cette commande **vérifie si le compte local bmarley est activé ou désactivé**.

Dans un second temps, si l'on souhaite réellement supprimer l'utilisateur.

```
Remove-LocalUser -Name "bmarley"
```

Cette commande supprime complètement le compte local **bmarley** de la machine.

```
Get-LocalUser
```



Quel est l'impact de cette suppression sur le dossier de profil de l'utilisateur s'il existe ?

Le **compte utilisateur est supprimé** de la liste des utilisateurs de Windows.

Le **dossier de profil** (souvent **C:\Users\bmarley**) **n'est pas toujours supprimé automatiquement**.

- Le dossier peut rester sur le disque, avec tous les fichiers personnels de l'utilisateur.
- Cela peut créer des fichiers "orphelins" si le compte n'existe plus.

Si tu veux supprimer également le dossier, il faut le faire **manuellement** ou via la commande **Remove-Item**.

Qu'est-ce qu'un SID Orphelin ?

SID (Security Identifier)

Chaque compte Windows possède un **identifiant unique appelé SID**.

Ce SID est utilisé pour gérer les permissions des fichiers, dossiers et ressources système.

SID orphelin

Se produit lorsque **le compte utilisateur est supprimé** mais que **ses fichiers ou dossiers existent encore**.

Les fichiers/dossiers restent sur le disque avec un **SID qui n'a plus de compte correspondant**.

Conséquence :

- Impossible d'identifier facilement le propriétaire.
- Les permissions peuvent devenir inutilisables ou compliquées à gérer.
- Risque de sécurité si le dossier contient encore des fichiers sensibles.

5 Scripter la création de comptes et l'ajout à un groupe

Attention ! Par défaut, Powershell bloque l'exécution de scripts. Afin d'exécuter des scripts, vous devez jouer la commande suivante :

Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy Unrestricted

Si vous devez créer 15 utilisateurs, les démarches de création d'utilisateur et d'ajout dans le/les groupes deviennent chronophages. Il est possible de créer un fichier avec une extension **.ps1** qui sera exécutable, un peu comme un fichier batch (c.f.r. cours de technologies des ordinateurs).

Mais avant de se lancer dans la création de multiples utilisateurs, il est intéressant de s'attarder à la création d'un script pour la création d'un simple utilisateur. Pour ce faire, imaginez que vous ayez besoin d'un script pour des utilisateurs temporaires, qui n'utiliseront que le système pour un temps limité (1 semaine par exemple). Par exemple, un membre de la filiale hollandaise nous rends visite en Belgique pour 1 semaine. Vous devez donc lui créer un compte local temporaire et il doit avoir des permissions très limitées et des règles de mot de passe spécifiques (différentes d'un employé normal).

Créez donc un script PowerShell nommé **"create_tmp_account.ps1"** qui doit effectuer les actions suivantes dans l'ordre :

Afficher un message, par exemple : **"Création du compte temporaire"**

Ensuite, demandez un mot de passe de manière sécurisée et le stocker dans une variable.

Créer un nouveau groupe local nommé **"Temp-Visitors"** avec la description **"Groupe pour les visiteurs temporaires, permissions limitées"**

Créer un nouvel utilisateur local nommé **"temp_external"** avec comme nom complet **"Externe Temporaire"**, une description **"Compte pour visiteur temporaire - Semaine 42"** et le mot de passe définit dans la variable.

Configurez la politique du compte **"temp_external"** avec :

- Le mot de passe ne doit jamais expirer (**PasswordNeverExpires**).

- L'utilisateur ne doit pas avoir le droit de changer son mot de passe (**UserMayChangePassword \$false**).

Ajouter l'utilisateur dans le groupe

Enfin, affichez les détails du compte (**Name, Enabled** et **PasswordNeverExpires** au minimum) et aussi les membres du groupe créé.

```
# =====
```

```
# Script : create_tmp_account.ps1
```

```
# Objectif : créer un compte utilisateur temporaire avec mot de passe sécurisé
```

```
# et permissions limitées pour un visiteur
```

```
# =====
```

```
# Afficher un message
```

```
Write-Host "Création du compte temporaire"
```

```
# Demander un mot de passe sécurisé
```

```
$Password = Read-Host -AsSecureString -Prompt "Entrez un mot de passe pour le  
compte temporaire"
```

```
# Créer les variables pour le nom du groupe et de l'utilisateur
```

```
$NomGroup = "Temp-Visitors"
```

```
$NomUtilisateur = "temp_external"
```

```
# Créer le groupe local
```

```
New-LocalGroup -Name $NomGroup -Description "Groupe pour les visiteurs  
temporaires"
```

```
Write-Host "Groupe '$NomGroup' créé."
```

```
# Créer le compte utilisateur local avec le mot de passe sécurisé
```

```
New-LocalUser -Name $NomUtilisateur -Password $Password -FullName "Externe  
Temporaire" -Description "Compte pour visiteur temporaire - Semaine 42"
```

```
Write-Host "Utilisateur '$NomUtilisateur' créé."
```

```
# Configurer les propriétés du compte utilisateur
```

```
Set-LocalUser -Name $NomUtilisateur -PasswordNeverExpires $True -
UserMayChangePassword $False

# Ajouter l'utilisateur au groupe local

Add-LocalGroupMember -Group $NomGroup -Member $NomUtilisateur

Write-Host "Utilisateur '$NomUtilisateur' ajouté au groupe '$NomGroup'."

# Afficher les détails du compte

Write-Host "Détails du compte $NomUtilisateur :"

Get-LocalUser -Name $NomUtilisateur | Select-Object Name, Enabled,
PasswordNeverExpires | Format-Table

# Afficher les membres du groupe

Write-Host "Membres du groupe '$NomGroup' :"

Get-LocalGroupMember -Group $NomGroup | Select-Object Name, ObjectClass |
Format-Table

# Fin du script

Write-Host "Script de création de compte temporaire terminé Bravo Jérémie Ns. Tu es
```

Toutes les commandes que j'ai utilisées dans mon script **ont déjà été vues dans les TP**, à l'exception de **Format-Table**.

Write-Host → affiche un message dans la console, déjà utilisé dans les TP pour informer l'utilisateur.

Read-Host -AsSecureString → permet de demander un mot de passe sans qu'il soit visible à l'écran.

New-LocalGroup → création d'un groupe local, vue dans les TP pour gérer les permissions.

New-LocalUser → création d'un utilisateur local avec mot de passe et description.

Set-LocalUser → configuration des propriétés du compte, comme mot de passe non expirant et interdiction de le changer.

Add-LocalGroupMember → ajout d'un utilisateur dans un groupe local.

Get-LocalUser / Get-LocalGroupMember → affichage des comptes et des membres de groupes.

La commande nouvelle : Format-Table

But : présenter la sortie de PowerShell **sous forme de tableau** avec des colonnes alignées pour chaque propriété sélectionnée.

Exécuter le script

Tu peux exécuter un script avec :

```
.\create_tmp_account.ps1
```

. indique que le script se trouve dans le **dossier courant**.

6 Livrables

Vous devez créer un script qui automatise toutes les manipulations vues dans ce TP. Le script devra créer les utilisateurs suivants :

tstark (Tony Stark), Description : **"Ingénieur en chef"**

nromanoff (Natasha Romanoff), Description : **"Responsable de la sécurité"**

pparker (Peter Parker), Description : **"Stagiaire R&D"**

Aussi, le script ne devra demander la saisie du mot de passe (sécurisée) qu'une seule fois et l'appliquer aux trois utilisateurs. Tous les utilisateurs devront changer leur mot de passe à la première connexion.

Le script devra créer deux groupes : Ingenierie et Securite.

- Ajouter tstark et pparker au groupe Ingenierie.
- Ajouter nromanoff au groupe Securite.

Le script devra se terminer en affichant la liste des membres de chaque groupe pour vérification.

7 Aide Powershell

7.1 Afficher en console

```
Write-Host "Bonjour et bienvenue !"
```

7.2 Créer un tableau d'objets en PS

```
$usersToCreate = @(
    @{Name="tstark"; FullName="Tony Stark"; Description="Ingénieur en chef"}
    @{Name="nromanoff"; FullName="Natasha Romanoff"; Description="Responsable
sécurité"}
    @{Name="pparker"; FullName="Peter Parker"; Description="Stagiaire R&D"}
)
```

7.3 Boucler sur un tableau en PS

```
foreach ($user in $usersToCreate) { ... }
```

7.4 Gérer les exceptions (ou les commandes conflictuelles) en PS

```
try { ... }
catch { ... }
```